

The AI/ML Interview Book

From Scratch to Pro — Theory, Code, and Interview Q&A

ch13_training

AYuSh

Contents

Chapter 13: Training Deep Networks	3
--	---

Chapter 13: Training Deep Networks

Chapter 12 built the machine; this chapter is about making it train — fast, stably, and to a solution that generalizes. Everything here is response to a failure mode: mini-batching answers "full gradients are too expensive," momentum and Adam answer "SGD crawls through ill-conditioned valleys," schedules and warmup answer "one learning rate can't both explore and converge," normalization answers "deep nets are exquisitely sensitive to scale," dropout and early stopping and augmentation answer "the network memorizes," mixed precision answers "memory and FLOPs bind," and Bayesian search answers "hyperparameters multiply." Interviews mine this chapter for compare-and-contrast questions — Adam vs AdamW, batch norm vs layer norm, grid vs random — where the winning answers name the mechanism, not just the preference.

Every claim runs: mini-batch beats full-batch per epoch by orders of magnitude on a fixed budget (Listing 1); six optimizers built from scratch race across an ill-conditioned valley (Listing 2); Adam's L2 pathology and AdamW's fix are isolated in a two-parameter experiment (Listing 3); a hot constant learning rate collapses to 52% while cosine + warmup reaches 97.8% (Listing 4); batch norm is implemented forward and backward and buys 20× learning-rate headroom (Listing 5); dropout and early stopping act on a staged overfit (Listing 6); ± 1 -pixel augmentation adds 3 points on small data (Listing 7); fp16 underflow is caught and repaired by loss scaling (Listing 8); grid, random, and Optuna's TPE compete on an equal 16-trial budget (Listing 9).

Gradient descent: batch, mini-batch, stochastic

The choice is how much data buys each parameter update. **Full-batch GD** computes the exact gradient over all n examples per step: maximally accurate steps, brutally expensive, and per *epoch* it learns almost nothing because one pass = one update. **SGD** (batch of 1) updates on every example: thousands of updates per epoch, each one a noisy estimate whose variance never lets the iterate settle. **Mini-batch** (32–512 typically) is the production compromise: enough samples for a usable gradient estimate, small enough for thousands of updates per epoch, and sized to saturate GPU parallelism. Listing 1 fixes the budget at five passes over 20,000 points: full-batch ends at MSE 20.06 (five exact steps go nowhere), mini-batch 256 reaches 0.254 — essentially the 0.251 noise floor — and batch-1 SGD lands slightly off (0.360) because its variance keeps it rattling around the optimum.

Three second-order facts interviewers reward. First, gradient noise scales as σ^2/B — halving the batch doubles gradient variance, which is why smaller batches need smaller learning rates (a common heuristic: scale LR linearly with batch size, with warmup covering the large-batch regime). Second, the noise is not purely a cost: it provides an implicit regularization/exploration effect, and very large batches famously tend toward

sharper minima that can generalize slightly worse — one reason "just use the biggest batch that fits" isn't free. Third, an **epoch** (full pass) and an **update** are different currencies: mini-batching wins because it converts the same data into vastly more updates.

Optimizers: momentum to Adam

Plain SGD's enemy is **ill-conditioning**. On a valley whose curvature differs by $100\times$ between directions (condition number 100), the stable learning rate is set by the *steepest* direction ($\eta < 2/L$), so the flat direction crawls; nudge the LR 15% past the edge and divergence is immediate (Listing 2: 3.5×10^{-4} at $\eta = 0.018$, 1.8×10^{18} at 0.021). Every optimizer after SGD is a workaround.

Momentum accumulates a velocity $v \leftarrow \beta v + g$ and steps along v : oscillations across the valley cancel in the running sum while progress along it compounds — like a heavy ball rolling through. Listing 2: 5.5×10^{-8} , four orders better than SGD at the same LR.

Nesterov (NAG) evaluates the gradient at the *look-ahead* point $w - \eta\beta v$ — a cheap correction that anticipates the landing spot and permits provably better convergence on convex problems (3.2×10^{-11} in the race). **Adagrad** divides each coordinate's step by $\sqrt{\sum g^2}$ accumulated over *all* history: brilliant for sparse features (rare coordinates keep big steps), fatal for deep nets — the denominator only grows, so the effective LR decays to zero mid-training. **RMSprop** fixes that with an exponential moving average $E[g^2] \leftarrow \rho E[g^2] + (1-\rho)g^2$: the denominator tracks *recent* gradient scale and can shrink again. **Adam** = RMSprop + momentum + bias correction: first and second moment EMAs m, v , each divided by $1 - \beta^t$ to undo the zero-initialization bias in early steps, update $w \leftarrow w - \eta \hat{m}/(\sqrt{\hat{v}} + \epsilon)$. It is the default because it is robust to LR choice across problems — in the race it converges at $\eta = 0.5$, $25\times$ SGD's stability edge, because per-coordinate rescaling makes the condition number nearly irrelevant.

AdamW fixes a subtle bug in how everyone used Adam. Weight decay implemented as L2-in-the-loss adds λw to the gradient — which Adam then divides by $\sqrt{\hat{v}}$, so *coordinates with large gradient history get almost no regularization*. Listing 3 isolates it: two weights facing pure zero-mean data gradients, one high-variance and one low-variance; Adam-with-L2 decays them to 0.944 vs 0.017 — a $55\times$ disparity in applied regularization — while AdamW, which subtracts $\eta\lambda w$ *outside* the adaptive machinery, decays both to exactly 0.804. That decoupling is why AdamW is the transformer-era default and why "Adam vs AdamW" is a favorite interview discriminator: the answer is one sentence — *L2 gets scaled by the adaptive denominator; decoupled decay doesn't*.

Learning rate schedules

The learning rate is the single most important hyperparameter, and its best value changes during training: early on you want large steps (fast progress, exploration), at the

end small ones (settle into the minimum instead of orbiting it). Listing 4 stages the failure of refusing to choose: a hot constant LR trains to only 52.2% test accuracy — permanently rattling — while the same LR under any decaying schedule exceeds 97%.

Step decay ($\div 10$ at fixed milestones) is the classic CNN recipe: simple, effective, but the milestones are two more hyperparameters and the loss curve shows cliff-shaped drops at each cut. **Cosine annealing** $\eta_t = \eta_0 \cdot \frac{1}{2}(1 + \cos(\pi t/T))$ decays smoothly to zero with no milestones to tune — the modern default (97.6% in the listing). **Cyclical/restart schedules** (SGDR) ramp the LR back up periodically, escaping to explore before re-converging; the related **one-cycle** policy (up then down within a single run) is popular for fast training.

Warmup — ramping linearly from ~ 0 to η_0 over the first few percent of steps — exists because the start of training is the most fragile moment: weights are random, early gradients are large and unrepresentative, and Adam's moment estimates are built from a handful of samples (its bias correction amplifies early noise). Large-batch and transformer training are especially warmup-dependent; skipping it is a classic source of "my transformer diverged in the first 500 steps." In Listing 4, cosine + 5% warmup is the best of the four (97.8%) and survives a starting LR that destroys the constant run. The reflex to state in interviews: **decay for the end, warmup for the beginning, and the LR-vs-loss curve (LR range test) to pick η_0 .**

Normalization: batch, layer, group

Batch normalization standardizes each feature over the mini-batch —

$\hat{z} = (z - \mu_B) / \sqrt{\sigma_B^2 + \epsilon}$ — then restores expressiveness with learned scale and shift $\gamma\hat{z} + \beta$.

Whatever the layers upstream do, each unit's pre-activations arrive centered and unit-scale, which tames the scale sensitivity that Chapter 12's init probes exposed and makes the loss surface effectively smoother. The measured payoff in Listing 5: a plain 5-layer MLP trains at LR 0.05 but collapses to chance at LR 0.3+; with from-scratch batch norm (forward *and* backward — the backward pass must route gradients through μ and σ too, which is the fiddly part) the same net scores 95–96% at LR 0.3 *and* 1.0 — roughly **20× learning-rate headroom**, which in practice converts to faster, more forgiving training. The original "internal covariate shift" explanation is now considered incomplete; the smoothing story has better evidence, and saying so signals current knowledge.

BN's cost is its **batch dependence**: at training time each example's output depends on its batchmates; at test time you must switch to running means/variances (a classic train/eval bug — forgetting `model.eval()`); tiny batches make the statistics noisy; and in distributed training the "batch" is per-device unless you pay for sync-BN. **Layer norm** removes the dependence by normalizing over *features within each sample* — identical computation at train and test, batch-size-1 friendly, sequence-friendly — which is why transformers and RNNs use it exclusively (Chapter 16). **Group norm** splits channels into groups and normalizes within each: batch-independent like LN, but preserving some of BN's per-channel behavior — the standard fallback for small-batch vision (detection/

segmentation with 1-2 images per GPU). Listing 5(a) shows the entire difference is *which axis you average over*. Selection rule: CNNs with decent batch sizes → BN; transformers/RNNs/any batch-sensitive setting → LN; small-batch vision → GN.

Regularization: dropout, early stopping, augmentation

Dropout zeroes each hidden unit independently with probability p during training, forcing redundancy — no unit can rely on a specific co-adapted partner that might vanish. The **inverted dropout** implementation detail matters and is a favorite interview probe: divide the surviving activations by $1 - p$ at training time, so test-time inference uses the network unchanged (no mask, no scaling). Interpretations: an implicit ensemble of 2^{units} subnetworks sharing weights, or noise injection on hidden representations. Listing 6 stages a memorization regime — 150 training digits, width 1024, train accuracy 1.000 by epoch 10 — where dropout 0.5 lifts test accuracy from 0.924 to 0.938. The gain is real and modest, which is the honest general lesson: dropout is one tool, strongest in large fully-connected layers; CNNs mostly replaced it with BN + augmentation, while transformers keep light dropout (~ 0.1) on attention and residuals.

Early stopping watches validation performance and halts when it stops improving (with a patience window), returning the best checkpoint. It is regularization: bounding the number of optimization steps bounds how far weights travel from their small init, behaving like an implicit L2 penalty. Listing 6's run peaks at epoch ~ 14 and drifts down after; patience-25 stopping banks 0.933 without touching any other knob. It is also free — you were computing validation metrics anyway — and composes with everything else, which is why it is near-universal in practice.

Data augmentation multiplies the training set with label-preserving transforms — the formal statement of an invariance you believe the task has. Listing 7: 200 training digits, adding random ± 1 -pixel shifts (zero-padded, since wrap-around `np.roll` corrupts an 8×8 image — a real bug caught during writing) lifts test accuracy from 0.926 to 0.956. Two lessons ride along: augmentation declares *domain knowledge* (shifts preserve digit identity; a vertical flip would turn 6 into 9 — wrong invariance, worse model), and it acts as regularization precisely because the network can no longer memorize a fixed input set. The modern vision stack (crops, flips, color jitter, mixup/cutmix, RandAugment) and NLP equivalents (back-translation, token masking) are this idea industrialized; Chapter 14 picks it up for CNNs.

Gradient clipping was demonstrated in Chapter 12 (norm 2979 → 5, direction preserved): in the training recipe it is the standard guard in RNN/transformer training against rare exploding mini-batches, applied after backward and before the optimizer step.

Mixed precision training

Training in float16 halves memory and unlocks tensor-core throughput; the catch is fp16's tiny envelope — max 65,504, smallest normal 6×10^{-5} , machine epsilon 10^{-3} (vs fp32's 10^{-7}). Listing 8 measures the two failure modes. **Gradient underflow**: late-training gradients of 2×10^{-8} flush to exactly 0.0 in fp16 — the update signal ceases to exist. The fix is **loss scaling**: multiply the loss by $S \sim 2^{10} - 2^{16}$ before backward (every gradient inherits the factor, lifting it into fp16's range), divide by S before the optimizer step; the listing recovers all four test gradients exactly. **Update swamping**: adding a 10^{-5} -sized update to a weight of 2.0 rounds to nothing in fp16 — Listing 8 applies 100 such updates and the fp16 weight moves 0.000000 while the fp32 copy moves correctly. The fix is keeping **fp32 master weights**: compute in fp16, update the fp32 copy, recast. The full AMP recipe = fp16/bf16 forward-backward + dynamic loss scaling + fp32 master weights + skip-step on inf/NaN. **bf16** (same 8-bit exponent as fp32, fewer mantissa bits) removes the underflow problem entirely — no loss scaling needed, at some precision cost — which is why bf16 is the large-model default on hardware that supports it (Chapter 29 continues into distributed training).

Hyperparameter search

With learning rate, schedule, weight decay, width, depth, dropout, and batch size all in play, search strategy matters. **Grid search** scales exponentially and — the Bergstra-Bengio argument — wastes its budget: with 16 trials over three hyperparameters, the grid tests only 4 distinct learning rates, and if LR is the only parameter that matters (it usually is), 16 trials bought 4 useful experiments. **Random search** draws every trial independently, so the same 16 trials test 16 distinct values of *every* hyperparameter — its projection onto the important dimension is $4\times$ denser for free. Listing 9 runs the comparison honestly on an equal budget: grid 0.9482, random 0.9494, and **Optuna's TPE** (Tree-structured Parzen Estimator — Bayesian optimization that models good-vs-bad trial densities and samples where good trials concentrate) 0.9516, finding an off-grid optimum (lr 0.0118, width 163) no grid would contain.

Practical craft the interviewer listens for: search in **log space** for LR, weight decay, and regularization strengths (the listing's `log=True` — a uniform grid in linear space wastes most points at the top of the range); use **successive halving / Hyperband / Optuna pruning** to kill bad trials early and reallocate budget; tune on validation data and reserve the test set for the final claim (Chapter 10's discipline); and quote the rough hierarchy — LR and schedule first, then weight decay and batch size, then architecture width/depth, then the rest. Bayesian methods shine when trials are expensive and the budget is tens-not-thousands; random remains the strong, parallel, embarrassingly simple baseline.

Code implementations

Listing 1 — Batch, mini-batch, and stochastic GD on a fixed budget

```

"""Listing 1: batch vs mini-batch vs stochastic gradient descent -- cost, noise, speed."""
import numpy as np, time
rng = np.random.default_rng(0)

n, d = 20_000, 50
X = rng.normal(size=(n, d)); w_true = rng.normal(size=d)
y = X @ w_true + rng.normal(0, 0.5, n)
loss = lambda w: np.mean((X @ w - y)**2)

def run(batch, lr, passes=5):
    w = np.zeros(d); t0 = time.time(); updates = 0
    for _ in range(passes):
        order = rng.permutation(n)
        for s in range(0, n, batch):
            idx = order[s:s+batch]
            g = 2 * X[idx].T @ (X[idx] @ w - y[idx]) / len(idx)
            w -= lr * g; updates += 1
    return loss(w), updates, time.time() - t0

print(f'variant:<22>{batch:>7}{final MSE:>11}{updates:>9}{time:>7}')
for name, batch, lr in [
    ("full-batch GD", n, 0.05),
    ("mini-batch 256", 256, 0.05),
    ("mini-batch 32", 32, 0.02),
    ("SGD (batch=1)", 1, 0.005)]:
    L, u, t = run(batch, lr)
    print(f'{name:<22}{batch:>7}{L:>11.4f}{u:>9}{t:>6.1f}s')
print(f'\nnoise floor (true w): {loss(w_true):.4f}')
print("same 5 passes over the data: mini-batches take thousands of cheap noisy steps"

"\nwhile full-batch takes 5 exact ones -- noise costs accuracy per step, wins per epoch")

```

Output:

variant	batch	final MSE	updates	time
full-batch GD	20000	20.0578	5	0.1s
mini-batch 256	256	0.2539	395	0.0s
mini-batch 32	32	0.2568	3125	0.0s
SGD (batch=1)	1	0.3596	100000	0.7s

noise floor (true w): 0.2507
same 5 passes over the data: mini-batches take thousands of cheap noisy steps
while full-batch takes 5 exact ones -- noise costs accuracy per step, wins per epoch

Five exact full-batch steps leave MSE at 20; the same five passes as mini-batches of 256 reach the 0.251 noise floor. Batch-1 SGD gets close but rattles (0.360) — gradient variance never lets it settle.

Listing 2 — Six optimizers from scratch on an ill-conditioned valley

```

"""Listing 2: six optimizers from scratch, raced on an ill-conditioned valley."""
import numpy as np

```



```

# f(w) = 0.5*(100*w1^2 + w2^2): condition number 100 -- the shape SGD hates
grad = lambda w: np.array([100*w[0], w[1]])
f = lambda w: 0.5*(100*w[0]**2 + w[1]**2)

def race(update, lr, steps=200, **st):
    w = np.array([1.0, 1.0]); state = {k: np.zeros(2) for k in st} | {"t": 0}
    for _ in range(steps):
        state["t"] += 1
        w = update(w, grad(w), lr, state)
    return f(w)

def sgd(w, g, lr, s): return w - lr*g
def momentum(w, g, lr, s):
    s["v"] = 0.9*s["v"] + g; return w - lr*s["v"]
def nag(w, g, lr, s):
    g_ahed = grad(w - lr*0.9*s["v"]); # Nesterov: gradient at the look-ahead point
    s["v"] = 0.9*s["v"] + g_ahed; return w - lr*s["v"]
def adagrad(w, g, lr, s):
    s["G"] += g**2; return w - lr*g/(np.sqrt(s["G"])+1e-8)
def rmsprop(w, g, lr, s):
    s["G"] = 0.99*s["G"] + 0.01*g**2; return w - lr*g/(np.sqrt(s["G"])+1e-8)
def adam(w, g, lr, s):
    s["m"] = 0.9*s["m"] + 0.1*g
    s["v"] = 0.999*s["v"] + 0.001*g**2
    mh = s["m"]/(1-0.9**s["t"]); vh = s["v"]/(1-0.999**s["t"]) # bias correction
    return w - lr*mh/(np.sqrt(vh)+1e-8)

print(f"{'optimizer':<10}{ 'lr':>7}{ 'f(w) after 200 steps':>22}")
for name, fn, lr, st in [("SGD", sgd, 0.018, {}),
                        ("SGD hot", sgd, 0.021, {}), # just past 2/L
                        ("Momentum", momentum, 0.018, {"v":0}),
                        ("NAG", nag, 0.010, {"v":0}),
                        ("Adagrad", adagrad, 0.5, {"G":0}),
                        ("RMSprop", rmsprop, 0.05, {"G":0}),
                        ("Adam", adam, 0.5, {"m":0, "v":0})]:
    print(f"{'name':<10}{lr:>7}{race(fn, lr, **st):>22.2e}")
print("\nSGD stable only for lr < 2/100; the steep axis sets the ceiling while the flat
axis crawls.")
print("Adaptive methods rescale per-coordinate -- condition number stops mattering.")

```

Output:

optimizer	lr	f(w) after 200 steps
SGD	0.018	3.50e-04
SGD hot	0.021	1.80e+18
Momentum	0.018	5.49e-08
NAG	0.01	3.23e-11
Adagrad	0.5	2.53e-96
RMSprop	0.05	1.26e-277
Adam	0.5	1.72e-08

SGD stable only for lr < 2/100; the steep axis sets the ceiling while the flat axis crawls.

Adaptive methods rescale per-coordinate -- condition number stops mattering.

SGD lives or dies within a 15% LR window; momentum gains four orders of magnitude by canceling oscillations; per-coordinate methods (Adagrad/RMSprop/Adam) run at 25× SGD's stability edge because they neutralize the condition number. (On this stationary quadratic Adagrad's aggressive decay is fine; on deep nets it strangles training mid-run — RMSprop/Adam exist for that reason.)

Listing 3 — Adam vs AdamW: the decoupling experiment

```

"""Listing 3: Adam vs AdamW -- why L2-in-the-loss breaks under adaptive scaling."""
import numpy as np
rng = np.random.default_rng(1)

# two weights, both should decay toward 0 equally (no data gradient at all here):
# w1 sees LARGE noisy gradients (frequent feature), w2 sees TINY ones (rare feature)
def train(decoupled, steps=2000, lr=1e-2, wd=1e-2):
    w = np.array([1.0, 1.0]); m = np.zeros(2); v = np.zeros(2)
    for t in range(1, steps+1):
        s = 1.0 if t % 2 else -1.0 # alternating sign: zero-
        mean, no drift
        g_data = np.array([5.0*s, 0.05*s])
        g = g_data if decoupled else g_data + wd*w # Adam: L2 folded into
        gradient
        m = 0.9*m + 0.1*g; v = 0.999*v + 0.001*g**2
        mh, vh = m/(1-0.9**t), v/(1-0.999**t)
        w = w - lr*mh/(np.sqrt(vh)+1e-8)
        if decoupled: w = w - lr*wd*w # AdamW: decay applied
    directly
    return w

wa, ww = train(False), train(True)
print("pure weight-decay task (data gradient alternates sign: zero mean, no drift):")
print(f"Adam + L2 in loss : w_large-grad {wa[0]:+.3f} w_small-grad {wa[1]:+.3f}")
print(f"    <- decay crushed where v is large")
print(f"AdamW (decoupled) : w_large-grad {ww[0]:+.3f} w_small-grad {ww[1]:+.3f}")
print(f"    <- uniform decay, as intended")
print("\nAdam divides EVERYTHING by sqrt(v) -- including the L2 term, so high-
variance")
print("coordinates get almost no regularization. AdamW subtracts lr*wd*w outside the")
print("adaptive machinery: every weight decays at the same relative rate.")

```

Output:

```

pure weight-decay task (data gradient alternates sign: zero mean, no drift):
Adam + L2 in loss : w_large-grad +0.944 w_small-grad +0.017 <- decay crushed
where v is large
AdamW (decoupled) : w_large-grad +0.804 w_small-grad +0.804 <- uniform decay, as
intended

Adam divides EVERYTHING by sqrt(v) -- including the L2 term, so high-variance
coordinates get almost no regularization. AdamW subtracts lr*wd*w outside the
adaptive machinery: every weight decays at the same relative rate.

```

Both weights should decay identically — their data gradients are zero-mean. Adam-with-L2 regularizes them 55× differently depending on gradient variance; AdamW decays both to exactly 0.804.

Listing 4 — Learning-rate schedules and warmup

```

"""Listing 4: learning-rate schedules -- constant vs step vs cosine, and why warmup
exists."""
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
rng = np.random.default_rng(2)

digits = load_digits(); X = digits.data/16.0

```

```

Xtr, Xte, ytr, yte = train_test_split(X, digits.target, test_size=0.25,
                                     random_state=0, stratify=digits.target)

def relu(z): return np.maximum(0,z)
def softmax(z):
    e = np.exp(z - z.max(1, keepdims=True)); return e/e.sum(1, keepdims=True)

def train(schedule, epochs=25, batch=64, lr0=1.2):
    Ws = [rng.normal(0, np.sqrt(2/m), (m,n)) for m,n in [(64,128),(128,10)]]
    bs = [np.zeros(128), np.zeros(10)]
    r = np.random.default_rng(0)
    total = epochs * (len(Xtr)//batch + 1); step = 0
    for ep in range(epochs):
        for s in range(0, len(Xtr), batch):
            step += 1
            lr = schedule(lr0, step, total)
            idx = r.permutation(len(Xtr))[s:s+batch]
            if not len(idx): continue
            H = relu(Xtr[idx] @ Ws[0] + bs[0]); P = softmax(H @ Ws[1] + bs[1])
            dz = P.copy(); dz[np.arange(len(idx)), ytr[idx]] -= 1; dz /= len(idx)
            dW1, db1 = H.T@dz, dz.sum(0)
            dz0 = (dz @ Ws[1].T) * (H > 0)
            Ws[1] -= lr*dW1; bs[1] -= lr*db1; Ws[0] -= lr*(Xtr[idx].T@dz0); bs[0] -
            =lr*dz0.sum(0)
            if not np.isfinite(Ws[0]).all(): return None
        H = relu(Xte @ Ws[0] + bs[0]); return (softmax(H@Ws[1]
        +bs[1]).argmax(1)==yte).mean()

schedules = {
    "constant (hot)": lambda lr0, t, T: lr0,
    "step /10 @ 60%,85%": lambda lr0, t, T: lr0 * (0.1 if t>0.85*T else (0.1 if
    t>0.6*T else 1.0)) if t<=0.85*T else lr0*0.01,
    "cosine": lambda lr0, t, T: lr0 * 0.5*(1+np.cos(np.pi*t/T)),
    "cosine + 5% warmup": lambda lr0, t, T: lr0 * min(t/(0.05*T), 1.0) *
    0.5*(1+np.cos(np.pi*max(t-0.05*T,0)/(0.95*T))),
}
for name, sch in schedules.items():
    acc = train(sch)
    print(f"{name:<24} test acc {'DIVERGED' if acc is None else f'{acc:.4f}'}")

```

Output:

```

constant (hot)          test acc 0.5222
step /10 @ 60%,85%     test acc 0.9733
cosine                  test acc 0.9756
cosine + 5% warmup     test acc 0.9778

```

The same initial LR that strands a constant schedule at 52% reaches 97.8% when decayed — the LR that explores well cannot also converge well. Warmup adds its increment by protecting the fragile first steps.

Listing 5 — Batch norm from scratch and the LR headroom it buys

```

"""Listing 5: batch norm from scratch -- what it computes, and the LR headroom it
buys."""
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
rng = np.random.default_rng(3)

# (a) the three norms differ only in WHICH axis they average over
x = rng.normal(2, 3, (8, 6)) # (batch, features)

```

```

bn = (x - x.mean(0)) / x.std(0)
# BatchNorm: stats per FEATURE, over batch
ln = (x - x.mean(1, keepdims=True)) / x.std(1, keepdims=True) # LayerNorm: per SAMPLE
print("axis of normalization batch-dependent? used in")
print("BatchNorm: over batch yes CNNs (needs batch stats at train)")
print("LayerNorm: over features no transformers, RNNs, batch=1 OK")
print("GroupNorm: feature groups no small-batch vision")
print(f"check: BN col means {np.round(bn.mean(0),8)[:3]} ... LN row means {np.round(ln.mean(1),8)[:3]}")

# (b) train deep MLP with/without BN at increasing LR
digits = load_digits(); X = digits.data/16.0
Xtr, Xte, ytr, yte = train_test_split(X, digits.target, test_size=0.25,
                                      random_state=0, stratify=digits.target)

def relu(z): return np.maximum(0,z)
def softmax(z):
    e=np.exp(z-z.max(1,keepdims=True)); return e/e.sum(1,keepdims=True)

def train(use_bn, lr, sizes=(64,128,128,128,10), epochs=30):
    r = np.random.default_rng(0)
    Ws=[r.normal(0,np.sqrt(2/m),(m,n)) for m,n in zip(sizes[:-1],sizes[1:])]
    bs=[np.zeros(n) for n in sizes[1:]]
    gam=[np.ones(n) for n in sizes[1:-1]]; bet=[np.zeros(n) for n in sizes[1:-1]]
    for ep in range(epochs):
        for s in range(0,len(Xtr),64):
            idx=r.permutation(len(Xtr))[s:s+64]
            if len(idx)<2: continue
            hs=[Xtr[idx]]; caches=[]
            for l in range(len(Ws)-1):
                z=hs[-1]@Ws[l]+bs[l]
                if use_bn:
                    mu,var=z.mean(0),z.var(0)+1e-5; zh=(z-mu)/np.sqrt(var)
                    caches.append((zh,var)); z=gam[l]*zh+bet[l]
            hs.append(relu(z))
            P=softmax(hs[-1]@Ws[-1]+bs[-1])
            dz=P.copy(); dz[np.arange(len(idx)),ytr[idx]]-=1; dz/=len(idx)
            dW=hs[-1].T@dz; db=dz.sum(0); dh=dz@Ws[-1].T
            Ws[-1]-=lr*dW; bs[-1]-=lr*db
            for l in range(len(Ws)-2,-1,-1):
                dz=dh*(hs[l+1]>0)
                if use_bn:
                    zh,var=caches[l]; n=len(idx)
                    dgam=(dz*zh).sum(0); dbet=dz.sum(0)
                    dzh=dz*gam[l]
                    dz=(dzh - dzh.mean(0) - zh*(dzh*zh).mean(0))/np.sqrt(var) # BN
            backward
            gam[l]-=lr*dgam; bet[l]-=lr*dbet
            dW=hs[l].T@dz; db=dz.sum(0); dh=dz@Ws[l].T
            Ws[l]-=lr*dW; bs[l]-=lr*db
            if not np.isfinite(Ws[0]).all(): return None
    # eval WITHOUT batch stats is sloppy; use full-train stats as running-stat stand-in
    h=Xtr; hs_te=Xte
    for l in range(len(Ws)-1):
        z=h@Ws[l]+bs[l]; zt=hs_te@Ws[l]+bs[l]
        if use_bn:
            mu,var=z.mean(0),z.var(0)+1e-5
            z=gam[l]*(z-mu)/np.sqrt(var)+bet[l]; zt=gam[l]*(zt-mu)/np.sqrt(var)+bet[l]
        h=relu(z); hs_te=relu(zt)
    return (softmax(hs_te@Ws[-1]+bs[-1]).argmax(1)==yte).mean()

print(f"\n{'lr':>6}{'plain 5-layer':>15}{'with BatchNorm':>16}")
for lr in [0.05, 0.3, 1.0]:
    a,b = train(False,lr), train(True,lr)
    print(f"{'lr':>6}{'DIVERGED' if a is None else f'{a:.4f}':>15}{'DIVERGED' if b is None else f'{b:.4f}':>16}")

```

Output:

	axis of normalization	batch-dependent?	used in
BatchNorm:	over batch	yes	CNNs (needs batch stats at train)
LayerNorm:	over features	no	transformers, RNNs, batch=1 OK
GroupNorm:	feature groups	no	small-batch vision
check:	BN col means [0. -0. -0.]	...	LN row means [-0. 0. 0.]

lr	plain 5-layer	with BatchNorm
0.05	0.9689	0.9622
0.3	0.5044	0.9511
1.0	0.1000	0.9644

BN/LN/GN differ only in the normalization axis. The training experiment is the point: the plain 5-layer net collapses at LR 0.3 and hits chance at 1.0, while the batch-normed net holds 95–96% throughout — 20× LR headroom, implemented in ~10 extra lines including the backward pass through the batch statistics.

Listing 6 — Inverted dropout and early stopping on a staged overfit

```

"""Listing 6: inverted dropout + early stopping -- regularization measured on a small-
data overfit."""
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
rng = np.random.default_rng(4)

digits = load_digits(); X = digits.data/16.0
Xtr, Xte, ytr, yte = train_test_split(X, digits.target, test_size=0.5,
                                     random_state=0, stratify=digits.target)
Xtr, ytr = Xtr[:150], ytr[:150] # tiny training set -> overfitting
regime
def relu(z): return np.maximum(0,z)
def softmax(z):
    e=np.exp(z-z.max(1,keepdims=True)); return e/e.sum(1,keepdims=True)

def train(p_drop, epochs=250, lr=0.3, width=1024, patience=None):
    r=np.random.default_rng(0)
    W1=r.normal(0,np.sqrt(2/64),(64,width)); b1=np.zeros(width)
    W2=r.normal(0,np.sqrt(2/width),(width,10)); b2=np.zeros(10)
    best, best_ep, wait = 0.0, 0, 0
    hist = {}
    for ep in range(epochs):
        for s in range(0,len(Xtr),32):
            idx=r.permutation(len(Xtr))[s:s+32]
            H=relu(Xtr[idx]@W1+b1)
            if p_drop:
                mask=(r.random(H.shape)>p_drop)/(1-p_drop) # INVERTED dropout: scale
                H=H*mask
            P=softmax(H@W2+b2)
            dz=P.copy(); dz[np.arange(len(idx)),ytr[idx]]-=1; dz/=len(idx)
            dW2=H.T@dz; dB2=dz.sum(0)
            dh=dz@W2.T
            if p_drop: dh=dh*mask
            dz1=dh*(relu(Xtr[idx]@W1+b1)>0)
            W2-=lr*dW2; b2-=lr*dB2; W1-=lr*(Xtr[idx].T@dz1); b1-=lr*dz1.sum(0)
        # test-time: NO mask, NO scaling (inverted dropout already handled it)
        tr_acc=(softmax(relu(Xtr@W1+b1)@W2+b2).argmax(1)==ytr).mean()
        te_acc=(softmax(relu(Xte@W1+b1)@W2+b2).argmax(1)==yte).mean()
        if ep in (9, 49, epochs-1): hist[ep+1]=(tr_acc,te_acc)
        if patience is not None: # early stopping on test-as-val
            if te_acc > best: best, best_ep, wait = te_acc, ep, 0

```

```

        else:
            wait += 1
            if wait >= patience: return hist, (best, best_ep)
    return hist, (best, best_ep)

for p in [0.0, 0.5]:
    hist, _ = train(p)
    line = " ".join(f"ep{e}: train {a:.3f}/test {b:.3f}" for e,(a,b) in hist.items())
    print(f"dropout p={p}: {line}")
_, (best, ep) = train(0.0, patience=25)
print(f"\nearly stopping (patience 25): best test {best:.3f} at epoch {ep+1} -- "
      "stops the overfit curve near its peak without any other regularizer")

```

Output:

```

dropout p=0.0: ep10: train 1.000/test 0.909  ep50: train 1.000/test 0.927  ep250: train
1.000/test 0.924
dropout p=0.5: ep10: train 1.000/test 0.928  ep50: train 1.000/test 0.928  ep250: train
1.000/test 0.938

early stopping (patience 25): best test 0.933 at epoch 28 -- stops the overfit curve
near its peak without any other regularizer

```

With 150 examples and width 1024 the network memorizes by epoch 10. Dropout 0.5 buys 1.4 test points at convergence; early stopping banks nearly the same for free. Note the inverted-dropout details: mask scaled by $1/(1-p)$ at train, applied to the backward pass too, and *no* scaling at test.

Listing 7 — Data augmentation: declaring an invariance

```

"""Listing 7: data augmentation -- label-preserving shifts beat extra epochs on small
data."""
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
rng = np.random.default_rng(5)

digits = load_digits(); X = digits.data/16.0
Xtr, Xte, ytr, yte = train_test_split(X, digits.target, test_size=0.5,
                                     random_state=0, stratify=digits.target)
Xtr, ytr = Xtr[:200], ytr[:200] # small-data regime

def shift(img8x8, dx, dy):
    """Shift with zero padding (np.roll would wrap pixels to the opposite edge)."""
    im = img8x8.reshape(8,8); out = np.zeros_like(im)
    src_y = slice(max(0,-dy), 8-max(0,dy)); dst_y = slice(max(0,dy), 8+min(0,dy))
    src_x = slice(max(0,-dx), 8-max(0,dx)); dst_x = slice(max(0,dx), 8+min(0,dx))
    out[dst_y, dst_x] = im[src_y, src_x]
    return out.ravel()

def relu(z): return np.maximum(0,z)
def softmax(z):
    e=np.exp(z-z.max(1,keepdims=True)); return e/e.sum(1,keepdims=True)

def train(augment, epochs=150, lr=0.3):
    r=np.random.default_rng(0)
    W1=r.normal(0,np.sqrt(2/64),(64,256)); b1=np.zeros(256)
    W2=r.normal(0,np.sqrt(2/256),(256,10)); b2=np.zeros(10)
    for ep in range(epochs):
        Xb, yb = Xtr, ytr
        if augment:

```

```

# originals + fresh random shifts each epoch
sh = r.integers(-1, 2, (len(Xtr), 2))
Xs = np.array([shift(x, *s) for x, s in zip(Xtr, sh)])
Xb, yb = np.vstack([Xtr, Xs]), np.concatenate([ytr, ytr])
for s in range(0, len(Xb), 32):
    idx=r.permutation(len(Xb))[s:s+32]
    H=relu(Xb[idx]@W1+b1); P=softmax(H@W2+b2)
    dz=P.copy(); dz[np.arange(len(idx)),yb[idx]]-=1; dz/=len(idx)
    dh=(dz@W2.T)*(H>0)
    W2-=lr*H.T@dz; b2-=lr*dz.sum(0); W1-=lr*Xb[idx].T@dh; b1-=lr*dh.sum(0)
return (softmax(relu(Xte@W1+b1)@W2+b2).argmax(1)==yte).mean()

print(f"200 training digits, 150 epochs:")
print(f" no augmentation          test acc {train(False):.4f}")
print(f" random +/-lpx shifts      test acc {train(True):.4f}")
print(f"\naugmentation = declaring an invariance (shifted digit -> same class);")
print(f"it multiplies effective data without new labels. Wrong invariances hurt:")
print(f"a vertical flip would turn 6 into 9.")

```

Output:

```

200 training digits, 150 epochs:
  no augmentation          test acc 0.9255
  random +/-lpx shifts      test acc 0.9555

augmentation = declaring an invariance (shifted digit -> same class);
it multiplies effective data without new labels. Wrong invariances hurt:
a vertical flip would turn 6 into 9.

```

Three test points from one pixel of shift — no new labels, just the knowledge that position doesn't change a digit's class. The zero-padded shift matters: wrap-around rolling (the lazy implementation) *hurt* accuracy in an earlier draft of this listing.

Listing 8 — Mixed precision: underflow, swamping, and the AMP recipe

```

"""Listing 8: mixed precision -- fp16 underflow, loss scaling, and master weights."""
import numpy as np

print(f"float16 range: max {np.finfo(np.float16).max}, tiniest normal\n{np.finfo(np.float16).tiny}")
print(f"float16 machine epsilon: {np.finfo(np.float16).eps}    (fp32:\n{np.finfo(np.float32).eps})\n")

# (a) gradient underflow: typical late-training gradients vanish in fp16
grads_fp32 = np.array([3e-5, 1e-6, 2e-8, 5e-9], dtype=np.float32)
print("gradient      fp32      fp16      fp16 after x1024 scaling")
for g in grads_fp32:
    scaled = np.float16(g * 1024.0)
    print(f" {g:.0e}    {np.float32(g):.2e}    {np.float16(g):>.2e}\n{np.float32(scaled)/1024:.2e}")

# (b) update swamping: w + lr*g rounds to w when the update is < eps*w
w32, g, lr = np.float32(2.0), 1e-4, 0.1
w16 = np.float16(2.0)
for _ in range(100):
    w32 = np.float32(w32 - lr*g)
    w16 = np.float16(w16 - np.float16(lr*g))
print(f"\n100 updates of size {lr*g:.0e} on w=2.0:")
print(f" fp32 master weights: {w32:.6f} (moved {2.0-w32:.6f})")
print(f" fp16 weights      : {w16:.6f} (moved {2.0-float(w16):.6f}) <- every update\nrounded away")

```



```
# (c) the AMP recipe
print("""
mixed-precision recipe (what torch.cuda.amp does):
  1. forward + backward in fp16/bf16      -> 2x memory, tensor-core speed
  2. loss * S before backward (S~2^10-2^16), grads / S after  -> no underflow
  3. fp32 MASTER copy of weights for the update                -> no swamping
  4. skip step if grads contain inf/nan (dynamic S adjusts)
bf16: same exponent range as fp32 -> no scaling needed, less mantissa precision.""")
```

Output:

```
float16 range: max 65504.0, tiniest normal 6.103515625e-05
float16 machine epsilon: 0.0009765625    (fp32: 1.1920928955078125e-07)

gradient      fp32      fp16      fp16 after x1024 scaling
3e-05   3.00e-05   3.00e-05   3.00e-05
1e-06   1.00e-06   1.01e-06   1.00e-06
2e-08   2.00e-08   0.00e+00   2.00e-08
5e-09   5.00e-09   0.00e+00   5.01e-09

100 updates of size 1e-05 on w=2.0:
  fp32 master weights: 1.998999 (moved 0.001001)
  fp16 weights        : 2.000000 (moved 0.000000)  <- every update rounded away

mixed-precision recipe (what torch.cuda.amp does):
  1. forward + backward in fp16/bf16      -> 2x memory, tensor-core speed
  2. loss * S before backward (S~2^10-2^16), grads / S after  -> no underflow
  3. fp32 MASTER copy of weights for the update                -> no swamping
  4. skip step if grads contain inf/nan (dynamic S adjusts)
bf16: same exponent range as fp32 -> no scaling needed, less mantissa precision.
```

Small gradients flush to exactly zero in fp16 and are fully recovered by $\times 1024$ loss scaling; small updates onto large weights round away entirely unless an fp32 master copy takes them. bf16 trades mantissa for fp32's exponent range and skips the scaling dance.

Listing 9 — Grid vs random vs Optuna on an equal budget

```
"""Listing 9: hyperparameter search -- grid vs random vs Optuna's TPE on the same
budget."""
import numpy as np, optuna, warnings
warnings.filterwarnings("ignore"); optuna.logging.set_verbosity(optuna.logging.WARNING)
from sklearn.datasets import load_digits
from sklearn.model_selection import cross_val_score
from sklearn.neural_network import MLPClassifier
rng = np.random.default_rng(6)

digits = load_digits(); X, y = digits.data/16.0, digits.target
def score(lr, alpha, width):
    m = MLPClassifier((int(width),), learning_rate_init=lr, alpha=alpha,
                      max_iter=60, random_state=0)
    with warnings.catch_warnings():
        warnings.simplefilter("ignore")
        return cross_val_score(m, X, y, cv=3, n_jobs=-1).mean()

BUDGET = 16
# grid: 16 trials cover only 4x2x2 values -- lr gets FOUR distinct values
lrs = np.logspace(-4, -1, 4); alphas = np.logspace(-5, -2, 2); widths = [32, 128]
grid = max(score(l, a, w) for l in lrs for a in alphas for w in widths)

# random: 16 trials give 16 DISTINCT values of every hyperparameter
```



```

best_r = 0
for _ in range(BUDGET):
    best_r = max(best_r, score(10**rng.uniform(-4,-1), 10**rng.uniform(-5,-2),
                             int(2**rng.uniform(5,8))))

# Optuna TPE: adaptive -- spends later trials near earlier good regions
def objective(t):
    return score(t.suggest_float("lr", 1e-4, 1e-1, log=True),
                 t.suggest_float("alpha", 1e-5, 1e-2, log=True),
                 t.suggest_int("width", 32, 256, log=True))
study = optuna.create_study(direction="maximize",
                           sampler=optuna.samplers.TPESampler(seed=0))
study.optimize(objective, n_trials=BUDGET)

print(f"same budget of {BUDGET} trials, 3-fold CV accuracy:")
print(f" grid search  best {grid:.4f}    (only 4 distinct lr values tested)")
print(f" random search best {best_r:.4f}    (16 distinct values per hyperparameter)")
print(f" Optuna (TPE) best {study.best_value:.4f}    params {dict((k, round(v,5) if
isinstance(v,float) else v) for k,v in study.best_params.items())}")
print("\nBergstra-Bengio: when few hyperparameters matter, random dominates grid
because")
print("grid wastes its budget re-testing the same values of the important dimension.")

```

Output:

```

same budget of 16 trials, 3-fold CV accuracy:
 grid search  best 0.9482    (only 4 distinct lr values tested)
 random search best 0.9494    (16 distinct values per hyperparameter)
 Optuna (TPE) best 0.9516    params {'lr': 0.01182, 'alpha': 9e-05, 'width': 163}

Bergstra-Bengio: when few hyperparameters matter, random dominates grid because
grid wastes its budget re-testing the same values of the important dimension.

```

Same 16 trials: the grid could only ever test 4 learning rates; random tested 16; TPE spent its later trials refining the region its early trials found, landing off-grid at lr 0.0118, width 163.

Pitfalls, comparisons and practical tips

Optimizer cheat table:

Optimizer	State kept	Key idea	Failure mode	Default when
SGD	none	raw gradient	ill-conditioning; LR knife-edge	with momentum + schedule: CNNs
Momentum	v	EMA of gradients	overshoot at high β	classic vision recipes
NAG	v	gradient at look-ahead	tighter stability range	marginal upgrade to momentum
Adagrad	Σg^2	per-coord decay, full history	LR $\rightarrow 0$ mid-training	sparse features/embeddings

Optimizer	State kept	Key idea	Failure mode	Default when
RMSprop	EMA g^2	forgetting denominator	needs tuning; no momentum	RNNs (historical)
Adam	m, v	RMSprop + momentum + bias corr.	L2 coupling; sometimes worse minima than SGD	default everywhere
AdamW	m, v	decoupled weight decay	—	transformers; the modern default

Normalization selector: batch $\geq \sim 32$ and CNN $\rightarrow$ BatchNorm; transformer/RNN/variable batch $\rightarrow$ LayerNorm; small-batch vision $\rightarrow$ GroupNorm. Never forget running-stats eval mode for BN.

The recurring pitfalls:

- **One LR for the whole run.** The listing's 52%-vs-98% gap is the argument. Decay something (cosine default); warm up transformers and large batches.
- **Adam with weight_decay as L2.** In most frameworks that flag on plain Adam is the coupled version; use AdamW. The difference is invisible in code review and real in results.
- **Forgetting `model.eval()` / running stats.** BN in train mode at inference uses batch statistics — predictions change with batch composition; a classic silent bug.
- **Dropout at test time** (or forgetting the $1/(1-p)$ train-time scale): systematically shrunk activations. Inverted dropout makes inference a no-op; verify by checking train/eval output scale agreement.
- **Tuning on the test set.** Schedule milestones, patience, and search all fit the data they see; keep validation and test separate (Chapter 10).
- **Linear-space hyperparameter grids.** LR/decay live on log scales; a linear grid spends 90% of its points in the top decade.
- **Augmenting the wrong invariance.** Flips on digits/text, hue shifts on medical stains — encode task knowledge, don't cargo-cult recipes. And check augmentation on top of originals; replacing them entirely can hurt (Listing 7's first draft did).
- **fp16 without loss scaling.** Loss plateaus late in training because gradients silently underflow to zero (Listing 8); bf16 or dynamic scaling.
- **Early stopping with patience 1.** Validation noise triggers premature stops; patience should absorb metric jitter (10–30 evals typical).
- **Trusting a single seed.** Chapter 10's lesson applies to training tricks: a 0.3-point "improvement" from a new optimizer needs seed-variance context before you believe it.

Interview questions and answers

Q1. Compare batch, mini-batch, and stochastic gradient descent. Why is mini-batch the standard?

Full-batch: exact gradient, one expensive update per pass — Listing 1 shows 5 exact steps leaving MSE at 20 while mini-batches reached the noise floor on the same budget. SGD (batch 1): maximal updates, but gradient variance keeps the iterate rattling (0.360 vs 0.254 floor). Mini-batch: enough samples for a useful gradient, thousands of updates per epoch, and matches GPU parallelism. Gradient noise scales as σ^2/B , linking batch size to feasible learning rate.

Q2. Why does plain SGD struggle on ill-conditioned problems?

Stability requires $\eta < 2/L$ where L is the largest curvature; progress along the flattest direction goes as $\eta \cdot \mu$. With condition number $\kappa = L/\mu$, the steep direction caps the LR while the flat one crawls — iterations scale with κ . Listing 2: at $\kappa=100$, SGD converges at $\eta=0.018$, explodes at 0.021. Momentum cancels cross-valley oscillation; adaptive methods rescale per-coordinate so κ nearly stops mattering (Adam stable at 25x SGD's edge).

Q3. Explain momentum and Nesterov momentum. What's the actual difference?

Momentum: $v \leftarrow \beta v + g$; $w \leftarrow w - \eta v$ — an EMA of gradients; oscillating components cancel, consistent components compound (like a heavy ball). Nesterov evaluates the gradient at the look-ahead point $w - \eta\beta v$ instead of w : it corrects using where the momentum step will land, giving a tighter effective correction and better convergence guarantees on convex problems. Practically NAG tolerates slightly different LRs and often converges a bit faster; conceptually: momentum = gradient then jump, NAG = jump then gradient.

Q4. Walk through Adam's update rule and explain the bias correction.

$m \leftarrow \beta_1 m + (1 - \beta_1)g$; $v \leftarrow \beta_2 v + (1 - \beta_2)g^2$; $\hat{m} = m / (1 - \beta_1^t)$; $\hat{v} = v / (1 - \beta_2^t)$; $w \leftarrow w - \eta \cdot \hat{m} / (\sqrt{\hat{v}} + \epsilon)$. m is a momentum-style EMA of gradients, v an EMA of squared gradients giving per-coordinate step normalization. Both start at zero, so early EMAs are biased toward zero by factor $(1 - \beta^t)$; dividing by it removes the bias — without correction, early steps are far too small ($\beta_2=0.999$ means v is $\sim 0.1\%$ of true scale after one step).

Q5. Adam vs AdamW — what exactly is decoupled, and why does it matter?

Weight decay as L2-in-the-loss adds λw to the gradient, which Adam divides by $\sqrt{\hat{v}}$ — so coordinates with large gradient variance receive almost no decay, and rarely-updated ones get crushed. AdamW applies $w \leftarrow w - \eta \lambda w$ outside the adaptive scaling: uniform relative decay for all weights. Listing 3 isolates it: same decay setting yields 0.944-vs-0.017 residual weights under Adam+L2 (55x disparity) and 0.804/0.804 under AdamW. Consequence: with Adam, L2 and weight decay are not equivalent (they are for plain SGD), and AdamW's decay hyperparameter transfers across LR's much better.

Q6. Why did Adagrad fall out of use for deep nets, and how does RMSprop fix it?

Adagrad divides by $\sqrt{(\sum \text{all past } g^2)}$: the denominator is monotonically increasing, so the effective LR decays toward zero regardless of whether you've converged — mid-training paralysis on long runs. RMSprop replaces the sum with an EMA, so the denominator tracks *recent* gradient magnitude and can shrink when gradients do. Adagrad survives where its aggressive decay is a feature: sparse features/embedding tables, where rare coordinates keep large steps.

Q7. Why use a learning-rate schedule at all? Compare step decay and cosine annealing.

The optimal LR is non-stationary: big early (progress) and small late (settling); a constant hot LR orbits the minimum forever — Listing 4's 52% vs 98%. Step decay divides by ~ 10 at hand-picked milestones: effective, but the milestones are hyperparameters and the loss shows cliff drops. Cosine anneals smoothly to zero with only the horizon T to choose — fewer knobs, marginally better results in the listing (0.9756 vs 0.9733), and the modern default. Cyclical/restart variants re-explore; one-cycle compresses the whole pattern into one run.

Q8. What is warmup and why do transformers need it?

Linear ramp from ~ 0 to η_0 over the first few percent of steps. Early training is fragile: random weights produce large, unrepresentative gradients; Adam's m, v are estimated from a handful of samples and the bias correction *amplifies* the noisy early estimates; large batches remove the gradient noise that would otherwise mask a too-hot step. Transformers add sensitive attention/LN dynamics at init. Skipping warmup is the classic "diverged in the first 500 steps" post-mortem; Listing 4 shows warmup winning even on a small MLP.

Q9. Write batch norm's forward computation. What are gamma and beta for?

Per feature over the batch: $\mu = \text{mean}(z)$, $\sigma^2 = \text{var}(z)$, $\hat{z} = (z - \mu) / \sqrt{(\sigma^2 + \epsilon)}$, $\text{out} = \gamma \hat{z} + \beta$, with γ, β learned. Without them normalization would pin every pre-activation to zero-mean/unit-variance — a representational straitjacket (e.g. sigmoid confined to its linear zone). γ and β let the network undo or rescale the normalization where useful, so BN constrains the *optimization geometry* without constraining what functions are expressible. At inference, μ and σ^2 come from training-time running averages.

Q10. Why does batch norm let you use much larger learning rates?

It re-standardizes each layer's pre-activations regardless of what upstream weight updates did, breaking the multiplicative scale sensitivity that makes deep nets diverge (Chapter 12's propagation probes), and it demonstrably smooths the loss landscape (reduces gradient Lipschitz constant) so bigger steps stay safe. Listing 5: plain 5-layer collapses at LR 0.3 and hits chance at 1.0; the batch-normed version holds 95-96% at both — 20x headroom. The original "internal covariate shift" story is incomplete; the smoothness account has stronger evidence.

Q11. Batch norm vs layer norm vs group norm — axes, trade-offs, and where each is used.

BN: statistics per feature/channel across the batch — strongest for conv nets with decent batches; costs batch dependence (train/eval running-stats switch, noisy small-batch statistics, sync issues in distributed). LN: statistics per sample across features — batch-independent, identical train/test, works at batch 1 and on variable-length sequences; the transformer/RNN standard. GN: per sample within channel groups — batch-independent with some of BN's channel structure; small-batch vision default (detection/segmentation). Listing 5(a): the entire difference is the axis you average over.

Q12. A model performs differently in eval mode than train mode. Name the usual suspects.

(1) BN using batch statistics at inference (missing eval switch) — predictions depend on batch composition; (2) dropout still active at test; (3) BN running stats poorly estimated (too-short training, momentum too high, or distribution shift between train and inference data); (4) non-inverted dropout without test-time scaling. Diagnostic: compare outputs on the same input in both modes; BN/dropout are the only layers that should differ, and inverted dropout should make dropout's difference pure noise.

Q13. Explain inverted dropout precisely. Why scale at train time rather than test time?

Train: $\text{mask} \sim \text{Bernoulli}(1-p)$, $h \leftarrow h \cdot \text{mask} / (1-p)$; backward passes through the same mask. Test: nothing — no mask, no scale. Scaling by $1/(1-p)$ at train keeps $E[h]$ identical between train and test, so inference is a plain forward pass: no special-case code, no risk of forgetting the scale, and inference latency unchanged. Original (non-inverted) dropout scaled by $(1-p)$ at test — every deployment had to remember it; inverted moved the bookkeeping to training where the mask already lives.

Q14. Why is dropout equivalent to an ensemble, and why is it weaker in conv layers?

Each mini-batch trains a random subnetwork (one of 2^n units weight-sharing members); test-time expectation approximates averaging their predictions — an implicit ensemble. In conv layers adjacent activations are spatially correlated, so dropping individual pixels barely removes information (neighbors reconstruct it); structured variants (SpatialDropout dropping whole channels, DropBlock dropping patches) restore the effect. In practice CNNs lean on BN + augmentation; transformers keep light dropout (~ 0.1) on attention weights and residual branches.

Q15. Why is early stopping a regularizer and not just a convenience?

It bounds the number of gradient steps, which bounds how far weights travel from initialization — for linear models with small init this is formally equivalent to an L2 constraint whose strength scales inversely with training time. Intuition: gradient descent fits dominant, generalizable structure first and noise later; stopping early keeps the former. Listing 6: test peaks near epoch 14, drifts down after; patience-based stopping banks the peak. Practical detail: patience must exceed validation-metric jitter, and you restore the best checkpoint, not the last.

Q16. What makes a good data augmentation, and how can augmentation hurt?

A good augmentation is a label-preserving transform matching a true task invariance: shifts for digits (Listing 7: +3 points from $\pm 1\text{px}$), crops/flips for natural images, back-translation for text. Harm modes: wrong invariance (vertical flip maps 6→9; hue shift can erase medically meaningful color), leakage of test-style information, or replacing originals entirely with heavy transforms so the train distribution drifts from test — Listing 7's first draft (shift-only, wrap-around artifacts) *lowered* accuracy until originals were mixed back in and padding fixed.

Q17. Describe the failure modes of fp16 training and the AMP recipe that fixes them.

Underflow: gradients below $\sim 6\text{e-}5$ (normals) flush toward zero — Listing 8 shows $2\text{e-}8$ becoming exactly 0.0, silently stalling late training. Swamping: $w + \delta$ rounds to w when $\delta < \text{eps} \cdot w$ ($\text{eps} = 1\text{e-}3$ in fp16) — 100 updates moved an fp16 weight 0.000000. Overflow: activations/grads above 65504 → inf. Recipe: fp16/bf16 forward-backward; dynamic loss scaling (multiply loss by $S \approx 2^{10-2^{16}}$ pre-backward, divide grads post) against underflow; fp32 master weights against swamping; skip steps with inf/nan while the scaler adapts. bf16 has fp32's exponent range — no scaling needed — at lower mantissa precision.

Q18. Why does random search beat grid search? State the Bergstra-Bengio argument.

Performance is usually dominated by one or two hyperparameters. A grid of N trials over d dimensions tests only $N^{(1/d)}$ distinct values per dimension — Listing 9's 16-trial grid tried exactly 4 learning rates. Random sampling gives N distinct values in every dimension, so its projection onto the important one is maximally dense at zero extra cost. Same budget: grid 0.9482, random 0.9494, and TPE 0.9516 by concentrating later trials near earlier winners — at an off-grid point (lr 0.0118, width 163) no reasonable grid contains.

Q19. How does Bayesian optimization (e.g., Optuna's TPE) decide what to try next?

It maintains a surrogate over hyperparameter space from completed trials and picks points optimizing an acquisition rule balancing exploration and exploitation. TPE specifically: split trials into good (top quantile) and bad, fit density models $l(x)$ and $g(x)$ to each set, and sample candidates maximizing $l(x)/g(x)$ — try what looks like the good trials and unlike the bad. Add pruning (Hyperband/successive halving): kill trials whose learning curves lag, reallocating budget. Wins when trials are expensive and sequential; random remains the parallel-friendly baseline.

Q20. Which hyperparameters do you tune first, and on which scale?

Order by leverage: (1) learning rate — log scale, via LR range test or a coarse log sweep; (2) schedule/warmup; (3) weight decay — log scale; (4) batch size (with LR co-scaled); (5) width/depth/dropout; (6) the rest. Log scale for anything multiplicative (LR, decay, alpha): a linear grid wastes most points in the top decade (Listing 9 samples log-uniform). Tune on validation only; one final test-set evaluation (Chapter 10).

Q21. Training loss suddenly spikes to NaN mid-run. Diagnose.

Ordered checks: (1) exploding gradients — log grad norms; fix with clipping, lower LR, warmup; (2) fp16 overflow — switch to bf16 or check scaler skip-counts; (3) numerically unsafe ops — log/exp/softmax without max-subtraction or eps, division by tiny std (BN with batch 1-2); (4) LR too high after a schedule restart or scaler growth; (5) bad batch — corrupted inputs, labels out of range. The clipping guard plus stable-softmax habits (Chapter 12) prevent most; a NaN-check hook that dumps the offending batch localizes the rest.

Q22. Your validation accuracy oscillates wildly between epochs. Causes and fixes?

(1) LR too high for this stage — add/steepen decay; (2) small validation set — metric noise, not model noise (compute the binomial std: 1000 samples gives $\pm 1.5\%$ at $p=0.9$); (3) BN with small batches — noisy statistics, try GN/LN or bigger batch; (4) heavy augmentation randomness leaking into eval; (5) genuine instability from too-large batch-LR combination. Distinguish metric noise from training instability by re-evaluating a fixed checkpoint twice: same number = training instability; different = evaluation noise.

Q23. Linear scaling rule: you increase batch size 8x. What else changes?

Gradient variance drops 8x, so scale LR $\sim 8x$ to keep the same effective noise/step — with warmup to survive the now-huge early steps (Goyal et al.'s ImageNet-in-1-hour recipe). Watch for: fewer updates per epoch (may need more epochs), sharper-minima generalization effects at extreme batch sizes, and BN statistics improving while gradient noise regularization weakens. If LR scaling destabilizes, LARS/LAMB (layer-wise adaptive rates) is the large-batch tool. Serving the same wall-clock: more parallelism, not more epochs.

Q24. When would you still pick SGD+momentum over Adam/AdamW?

Vision CNNs where the well-tuned SGD recipe (momentum 0.9, step/cosine decay, weight decay $5e-4$) is known to reach slightly better final accuracy than Adam — the classic generalization-gap observation; when memory is tight (Adam keeps two extra states per parameter — for a 7B model that's 56GB extra in fp32); and when reproducing baselines that used it. AdamW when: transformers, sparse/noisy gradients, limited tuning budget, or anything where robustness-to-LR matters more than the last 0.3%.

Q25. What is gradient accumulation and when is it the right tool?

Run k forward/backward passes summing gradients before one optimizer step: emulates batch size $k \times B$ when memory allows only B . Details that bite: divide the loss (or grads) by k so the step matches a true large batch; BN statistics still see only B (a genuine difference — GN/LN unaffected); LR should follow the *effective* batch. Right tool for large-model fine-tuning on small GPUs and for matching a paper's batch size; wrong tool if the bottleneck is throughput rather than memory (it doesn't speed anything up).

Q26. How do dropout, weight decay, early stopping, and augmentation interact? Do you stack them all?

They regularize through different channels — noise on activations, penalty on weight magnitude, bound on optimization length, expansion of data — so they compose, but their strengths add: stacking all at full strength commonly underfits. Practice: augmentation is nearly always on (it encodes knowledge, not just noise); weight decay is cheap and default (AdamW ~ 0.01 - 0.1); early stopping is free insurance; dropout is the one most often reduced or dropped when BN/augmentation already suffice. Tune the total regularization budget against the train/val gap: gap large $\rightarrow$ add strength; train acc itself low $\rightarrow$ remove.

Q27. Interpret these training curves: train loss falling, validation loss flat at a high value from epoch 1.

Not classic overfitting (val would fall then rise). Suspects, in order: (1) validation pipeline bug — different preprocessing/normalization than train, wrong labels, eval-mode issues (BN/dropout); (2) distribution shift between train and val splits; (3) leakage making train trivially easy while val reflects the real task; (4) LR/schedule leaving the model memorizing noise immediately (tiny data, huge capacity). First move: evaluate the *training* data through the validation pipeline — if train accuracy is also bad there, the pipeline is the bug, not the model.

Q28. Design the full training recipe for a mid-size transformer from this chapter's pieces, and justify each choice.

AdamW (decoupled decay $\sim 0.05-0.1$) because adaptive per-coordinate scaling suits heterogeneous transformer gradients and L2-coupling is broken under Adam; LR $\sim 3e-4$ found by short log sweep, cosine decay to $\sim 10\%$ with 1-5% linear warmup (fragile early Adam estimates + large batch); LayerNorm not BN (batch-independence, sequence-friendly); dropout 0.1 on attention/residuals; global-norm clipping at 1.0 (rare exploding batches); bf16 mixed precision with fp32 master weights (no loss-scaling dance); gradient accumulation to the target effective batch; early stopping on validation loss with patience; random-then-TPE search over LR/decay/warmup if budget allows, log-scaled. Each element answers a named failure mode — that mapping is what the interviewer is scoring.